

TEORIA DA COMPUTAÇÃO E COMPILADORES

Rodrigo Fernandes da Silva

CONSTRUÇÃO DE UM COMPILADOR PARA A LINGUAGEM ARITHLANG

1. Introdução

Os compiladores são ferramentas fundamentais para a área de Computação, pois são responsáveis por transformar programas escritos em linguagens de alto nível em instruções que podem ser executadas por uma máquina. O estudo de compiladores permite compreender de forma mais profunda o funcionamento das linguagens de programação, bem como conceitos relacionados à análise de código, representação intermediária e execução de programas.

O objetivo deste trabalho foi desenvolver um compilador funcional para uma mini-linguagem denominada ArithLang. Apesar de seu escopo reduzido, a linguagem contempla os principais elementos necessários para demonstrar o funcionamento das etapas clássicas de um compilador, incluindo análise léxica, análise sintática, construção da Árvore Sintática Abstrata (AST), análise semântica, geração de código intermediário e execução por meio de uma máquina virtual baseada em pilha.

A linguagem implementada suporta declaração de variáveis, atribuição, números inteiros e reais, comentários, operações aritméticas básicas e o comando de saída print.

Além do desenvolvimento do compilador, foram elaborados testes automatizados para validar o funcionamento das diferentes fases do processo de compilação e execução.

2. Gramática Formal

A linguagem ArithLang foi implementada utilizando uma gramática baseada em EBNF.

```
program      → statement* EOF

statement    → letStmt
              | assignStmt
              | printStmt

letStmt      → 'let' IDENTIFIER '=' expression

assignStmt   → IDENTIFIER '=' expression

printStmt    → 'print' '(' expression ')'

expression   → term (('+' | '-') term)*

term         → unary (('*' | '/') unary)*

unary        → '-' unary
              | primary

primary      → INTEGER
              | FLOAT
              | IDENTIFIER
              | '(' expression ')'
```

A estrutura da gramática foi projetada para garantir automaticamente a precedência correta entre os operadores aritméticos. Dessa forma, multiplicações e divisões possuem maior precedência que adições e subtrações.

Exemplo:

2 + 3 * 4

Resultado:

14

Enquanto:

$$(2 + 3) * 4$$

Resultado:

20

3. Análise Léxica

A análise léxica é a primeira etapa do compilador. Seu objetivo é percorrer o código-fonte caractere por caractere e agrupar sequências de caracteres em unidades chamadas tokens.

Os tokens implementados foram:

Token	Descrição
INTEGER	Número inteiro
FLOAT	Número real
IDENTIFIER	Nome de variável
LET	Palavra-chave de declaração
PRINT	Palavra-chave de saída
PLUS	Operador +
MINUS	Operador -
STAR	Operador *
SLASH	Operador /
ASSIGN	Operador =
LPAREN	Parêntese esquerdo
RPAREN	Parêntese direito
NEWLINE	Quebra de linha
EOF	Final de arquivo

O Lexer também registra a linha e coluna de cada token, permitindo que mensagens de erro indiquem exatamente onde o problema ocorreu.

Exemplo de saída do Lexer:

```
let x = 10

LET
IDENTIFIER(x)
ASSIGN
INTEGER(10)
EOF
```

Comentários iniciados com "#" são ignorados até o final da linha.

4. Análise Sintática

Após a tokenização, os tokens são enviados ao Parser, implementado utilizando a técnica de descida recursiva.

Cada regra da gramática corresponde a um método específico do parser.

A principal responsabilidade desta etapa é verificar se a sequência de tokens segue a gramática da linguagem e construir a Árvore Sintática Abstrata (AST).

Exemplo:

Código:

```
let x = 2 + 3 * 4
```

AST produzida:

```
Program
├─ LetStatement
│   └─ BinaryOp(+)
│       ├── IntLiteral(2)
│       └─ BinaryOp(*)
│           ├── IntLiteral(3)
│           └─ IntLiteral(4)
```

A AST remove detalhes sintáticos desnecessários e preserva apenas a estrutura lógica do programa.

5. Análise Semântica

A análise semântica verifica regras que não podem ser expressas apenas pela gramática.

Foi implementada uma tabela de símbolos responsável por armazenar todas as variáveis declaradas durante a compilação.

As verificações realizadas incluem:

- Uso de variável sem declaração prévia.
- Re-declaração de variável.
- Atribuição para variável inexistente.

Exemplo de erro:

```
print(y)
```

Resultado:

Erro Semântico:

variável 'y' não declarada

A tabela de símbolos foi implementada utilizando um dicionário associando nome e tipo da variável.

Exemplo:

```
{  
    "x": "int",  
    "area": "float"  
}
```

Também foi implementada inferência simples de tipos.

Regras:

- `int + int → int`
- `float + float → float`
- `int + float → float`
- `divisão → float`

6. Geração de Código e Máquina Virtual

Após a validação semântica, a AST é percorrida para gerar instruções para uma máquina virtual baseada em pilha.

As principais instruções implementadas foram:

Instrução	Função
PUSH	Empilha valor
LOAD	Carrega variável
STORE	Armazena variável
ADD	Soma
SUB	Subtração
MUL	Multiplicação
DIV	Divisão
NEG	Negação
PRINT	Impressão
HALT	Encerramento

Exemplo:

Código:

```
let x = 2 + 3 * 4
```

Bytecode gerado:

```
PUSH 2  
PUSH 3  
PUSH 4  
MUL  
ADD  
STORE x
```

A execução ocorre em uma Máquina Virtual (VM), que mantém:

- Pilha de operandos
- Ambiente de variáveis
- Saída do programa

O modelo baseado em pilha simplifica a implementação e é amplamente utilizado em ambientes educacionais para estudo de compiladores.

7. Testes

Foram implementados 22 testes automatizados utilizando Pytest.

Os testes cobrem:

- Operações aritméticas
- Precedência de operadores
- Uso de parênteses
- Declaração de variáveis
- Atribuições
- Impressão de resultados
- Comentários
- Erros léxicos
- Erros sintáticos
- Erros semânticos
- Divisão por zero
- Geração correta de bytecode

Resultado obtido:

22 passed

Os testes foram fundamentais para validar o comportamento esperado do compilador durante o desenvolvimento.

8. Uso de Inteligência Artificial

Durante o desenvolvimento deste projeto foi utilizado o ChatGPT para algumas funcionalidades nas seguintes etapas:

- Estruturação inicial do analisador léxico (Lexer) e definição dos tokens da linguagem.
- Esclarecimento de dúvidas sobre a implementação do parser de descida recursiva e construção da AST.
- Sugestões para a implementação da análise semântica e da tabela de símbolos.
- Apoio na geração de bytecode e na implementação da máquina virtual baseada em pilha.
- Auxílio na elaboração e revisão dos testes automatizados.
- Suporte na identificação e correção de erros encontrados durante o desenvolvimento.

9. Conclusão

O desenvolvimento deste projeto permitiu compreender de forma prática as principais etapas envolvidas na construção de um compilador.

Mesmo utilizando uma linguagem simples, foi possível implementar conceitos fundamentais da área, incluindo tokenização, análise sintática, representação por árvores, validação semântica, geração de código intermediário e execução em máquina virtual.

Os resultados obtidos demonstram que o compilador é capaz de processar programas válidos da linguagem ArithLang, identificar erros e executar corretamente expressões aritméticas e manipulação de variáveis.

Como trabalhos futuros, poderiam ser adicionados novos recursos à linguagem, como suporte a strings, operadores adicionais, estruturas condicionais, laços de repetição e múltiplos escopos.